EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA

TECNOLÓGICO NACIONAL DE MÉXICO

Tecnológico Nacional de México

Campus Querétaro

E11-Desarrollador / Integrador

Producto de la asignatura:
Ingeniería de Software

Desarrolla por:

Hernandez Perrusquia Jhoel
21140780

Asesor docente:

Villeda Maldonado Julio Alejandro

Fecha de entrega:

26 de febrero de 2026

Grupo:
6A

Análisis técnico

1. Verificación de coherencia con E10 (Diagramas de secuencia)

La estructura de clases actual soporta de manera exacta el intercambio de mensajes y la lógica temporal modelada en las secuencias:

- **Soporte de Lógica de Negocio:** Los métodos **`validarCupo(cursoId)`** y **`verificarInscripcionPrevia(uId, cId)`** en la clase **`InscripcionService`** proporcionan la funcionalidad necesaria para ejecutar los fragmentos combinados **`alt`** del Caso 1. Esto garantiza que la decisión de "Cupo lleno" o "Inscripción previa" no sea solo un flujo visual, sino una operación de negocio respaldada por la arquitectura.
- **Gestión de flujo de certificado:** El método **`validarUsuario(noTrabajador)`** en **`ConstanciaService`** es el punto de entrada que permite la transición desde la interfaz hacia la lógica de validación de horas. La clase **`ConstanciaService`** integra el método **`verificarCursosCompletados()`**, que es el motor encargado de filtrar la lista de cursos aptos para descarga antes de invocar la generación del PDF.
- **Persistencia de Estados:** La inclusión de métodos como **`actualizarCupo(id, nuevoCupo)` en `CursoRepository`** refleja la activación final del flujo de inscripción, donde la base de datos debe reflejar el decremento de espacios disponibles tras un registro exitoso.

2. Identificación de clases críticas

Se han identificado tres componentes cuyo correcto funcionamiento determina la estabilidad del sistema:

- **InscripcionService:** Es el corazón del sistema. Centraliza las reglas que evitan el sobrecupo y la duplicidad de registros, mitigando los riesgos de inconsistencia por concurrencia detectados por el desarrollador.
- **ConstanciaService:** Clase crítica para la validez institucional. Su lógica depende de la integridad de los datos provistos por el instructor (RF-08, RF-09), asegurando que el PDF solo se genere si el cumplimiento es del 100%.
- **Inscripcion (Entity):** Es la entidad de relación más importante. Al funcionar como el nexo entre **`Usuario`** y **`Curso`**, permite la trazabilidad completa del historial académico y es la fuente de datos primaria para el motor de generación de constancias.

3. Análisis de acoplamiento

El diseño presenta un **Bajo Acoplamiento**, lo que facilita el mantenimiento futuro:

- **Encapsulamiento de Persistencia:** Los servicios no interactúan directamente con la tecnología de almacenamiento; dependen de las clases **`Repository`**. Esto permite que, si el motor de base de datos cambia en el futuro, la lógica de negocio en los servicios permanezca intacta.
- **Independencia de Entidades:** Las entidades **(`Usuario`, `Curso`, `Constancia`)** son objetos planos de datos (POJOs) que no conocen la existencia de controladores ni servicios. Esta independencia evita que cambios en la interfaz de usuario obliguen a modificar la estructura de los datos base.
- **Jerarquia de herencia:** El uso de especializacion en la clase **`Usuario` (`Docente`, `Administrativo`, `Instructor`)** permite que los controladores manejen un perfil genérico para el acceso, pero utilicen atributos específicos (como la contraseña proporcionada por DDA) solo cuando el rol lo requiere, reduciendo la interdependencia innecesaria de campos.

4. Análisis de cohesión

El sistema presenta una **Alta Cohesión**, cumpliendo con el principio de responsabilidad única (Single Responsibility Principle):

- **Especialización de Servicios: `InscripcionService`** se limita exclusivamente a validar y registrar inscripciones. No contiene lógica para generar documentos ni para gestionar usuarios, lo que facilita la detección de errores y el testing unitario.
- **Controladores de orquestacion:** Los controladores **(InscripcionController** y **ConstanciaControlller)** actúan únicamente como puentes. Su cohesión es alta porque se limitan a recibir la solicitud de la interfaz y dirigirla al servicio correspondiente, sin ejecutar reglas de negocio internas.
- **Entidades atómicas:** Cada entidad representa una única unidad de información de dominio. La clase **Inscripcion** contiene unicamente los datos de la relación, mientras que la clase **Constancia** se encarga solo de la información del documento final, manteniendo una estructura limpia y sin mezcla de propositos.